

Domain-Adapted LLMs for VLSI Design and Verification: A Case Study on Formal Verification

Mingjie Liu^{*1}, Minwoo Kang^{*1,2}, Ghaith Bany Hamad¹, Syed Suhaib¹, Haoxing Ren¹
¹ NVIDIA ² UC Berkeley

Abstract—Large language models (LLMs) present unprecedented opportunities in task automation for industrial chip design and verification that can yield significant improvements in engineering productivity. Instead of deploying off-the-shelf LLMs, we present our methodology for adapting a language model to the domain of VLSI design, and we show that our domain-adapted model, ChipNeMo, achieves improved performance against models of similar size on benchmarks concerning chip design and electronic design automation (EDA). We finally present a case study on the prospective of applying LLMs to hardware formal verification. Our results indicate that the largest and most capable models, such as GPT-4, are able to generate syntactically correct SVA implementations, yet there exists room for improvement in ensuring precise reflection of user intent given as high-level natural language descriptions of formal properties.

I. INTRODUCTION

Recent large language models (LLMs) exhibit remarkable capabilities in natural language understanding, code generation, and mathematical or algorithmic reasoning, at times reaching or even surpassing human-level performance [1], [2], [3], [4]. Importantly, LLMs today are capable of instruction-following and in-context learning, which combined with advances in prompting techniques [5] have enabled these models to reason about complex logical problems and utilize external tools via function calling. As a result, language models have been employed towards question-answering and task automation in various applications, including medicine [6], materials science [7], and electronic design automation (EDA) [8], [9].

However, grounding a language model on a specialized, technical domain presents non-trivial challenges. While these models are trained on a large corpus of text and code, we are often interested in deploying models that are further equipped with *domain-specific* knowledge and reasoning capabilities. To this end, prior work suggest pretraining a new foundation model from scratch only with domain-related data—examples include BloombergGPT (50B) [10] for finance and BioMedLM (2.7B) [11] for clinical medicine. Although these models outperform general natural language models in their respective domains, the computational and data requirements for full pretraining renders this approach generally unviable.

An attractive alternative is to instead adapt an off-the-shelf LLM trained on general text corpora by continued pretraining [12] or fine-tuning [13] on in-domain datasets. In fact, it is often advantageous to start with pretrained models provided

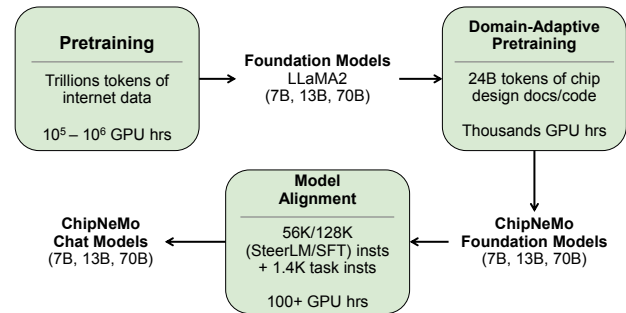


Fig. 1: The training flow for ChipNeMo [8] domain-adaptive pretraining (DAPT) and model alignment, including supervised fine-tuning (SFT).

that their existing capabilities such as natural language understanding and code generation are relevant for downstream tasks. Priors about commonsense and world knowledge also have been reported to underpin problem solving in novel domains [6], and hence increasing numbers of work take the approach of adapting pretrained foundation models via further parameter updates.

In this paper, we present our experience of training ChipNeMo [8], a language model adapted to the domain of VLSI chip design. We detail the techniques that we apply, including continued domain-adaptive pretraining (DAPT) and supervised fine-tuning (SFT). For automated evaluation, we also create a multiple-choice question-answering benchmark on tasks pertaining to EDA. Through experiments, we find that ChipNeMo models significantly out-perform LLaMA models that they were adapted from, across the entire suite of benchmarks.

Finally, given the success of ChipNeMo, we investigate the application of these models on an even more challenging task of hardware formal verification (FV). Recent work have initiated research on the use of LLMs in verification [14], [15], [16], a component of industrial chip design that is indispensable yet tedious and resource-consuming. In our case study, we explore the current capabilities of ChipNeMo and other generally available models, mark their limitations, and propose future research directions towards realizing the promises that LLMs present in the FV domain.

II. RELATED WORK

LLMs for Electronic Design Automation Researchers have started to apply LLMs to problems in chip design. Earlier work take existing models and evaluate their capabilities in performing tasks such as RTL code generation (e.g. Verilog)

^{*}Equal Contribution

Data Source Type	Data Percentage (%)	Data Tokens (B)	Training Percentage (%)	Training Tokens (B)
Bug Summary	9.5%	2.4	10.0%	2.4
Design Source	47.0%	11.9	24.5%	5.9
Documentation	17.8%	4.5	34.0%	8.2
Verification	9.1%	2.3	10.4%	2.5
Other	7.9%	2.0	12.0%	2.9
Wikipedia	5.9%	1.5	6.2%	1.5
Github	2.8%	0.7	3.0%	0.7
Total	100.0%	25.3	100.0%	24.1

TABLE I: Breakdown of DAPT data by source [8]. Token count measured with original LLaMA2 tokenizer.

[17], [9]. More recent work fine-tune open-source language models such as CodeGen [18] and LLaMA2 [2] on chip design related data and show that fine-tuned models out-perform both open-source and proprietary models in Verilog code generation [19], [20] and EDA tool script generation [21].

Language Models and ML for Formal Verification Prior work on applying LLMs to hardware formal verification can be categorized into largely two groups: one line of work take microarchitectural descriptions of the design-under-test (DUT) and prompt LLMs to generate a plausible formal testbench [16], [22], [14], [23]. Another line of work instead have LLMs consume architectural specification documents, describing high-level specifications that has not yet been formulated as RTL implementations, and have the models create an entire testbench [15]. Prior to the application of LLMs, several work have applied earlier techniques in natural language processing and machine learning at large [24], [25], [26].

In our case study, we administer evaluation of LLM capabilities by providing a formal testbench implementation that does not yet contain any formal properties, and directly have LLMs translate high-level user intent, describing the type of assertion to be created, into concrete SystemVerilog Assertion (SVA) implementation. We posit that this approach gives clearer understanding of language model performance as it closely checks if models are able to comprehend the semantics of formal properties and quantitatively evaluates whether LLMs exhibit capabilities beyond being able to simply create some number of syntactically correct assertions for a DUT.

III. CHIPNEMO

In this section, we discuss the training and evaluation of ChipNeMo [8], a large language model adapted to the VLSI chip design domain. We will describe the three main techniques that enable this adaptation: Domain-Adaptive Pre-training (DAPT), Domain-Adapted Tokenizer, and Model Alignment with Supervised Fine-Tuning (SFT), for which we provide an overview in Fig. 1.

A. Training ChipNeMo

1) *Domain-Adaptive Pretraining (DAPT)*: DAPT is a cost- and data-efficient alternative to pretraining a foundation model from scratch on domain-specific data. In our study, we apply DAPT on the LLaMA2 foundation base models with parameter sizes 7B, 13B, and 70B. The DAPT dataset is a combination of proprietary chip design data sources and publicly available datasets: the internal dataset includes documents pertinent to

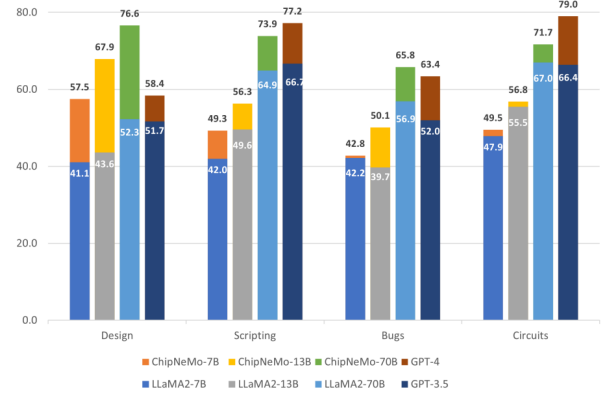


Fig. 2: ChipNeMo evaluation results on EDA-specific benchmark, compared against other LLMs. Results from [8].

chip design, verification, and infrastructure, which after pre-processing and cleaning amounts to 23.1 billion tokens; the public dataset includes both open-source documents on natural language and chip design related code repositories, mainly drawn from Wikipedia and GitHub data. Further summary of DAPT data sources are provided in Table I. All model training procedures are based on the NVIDIA NeMo framework [27].

2) *Domain-Adaptive Tokenizer*: Adapting a pre-trained model into a new domain requires extensions to the token vocabulary set since the target domain often contains technical terms or requires support for particular programming language syntax. In our study, we adapt the LLaMA2 tokenizer with the main goals of improving tokenization efficiency on domain-specific data, maintaining language model performance on general datasets, and at the same time minimizing the effort for future retraining/fine-tuning.

3) *Domain-Specific Fine-tuning and Model Alignment*: After DAPT, we perform model alignment with supervised fine-tuning (SFT). We adopt the identical hyperparameter training configuration as DAPT for all models, while only reducing the global batchsize. For SFT, we collect and train on about 128k general chat data.

B. Evaluation of ChipNeMo

For scalable and quantitative evaluation of various models, we devise a suite of multiple-choice question-answer (MCQA) benchmarks for a set of EDA-specific tasks, similar in format to established LLM benchmarks such as MMLU [28]. We follow general practice of including at least one challenging answer choice other than the ground-truth answer, so that the accuracy on the benchmark sufficiently reflects domain expertise of the models. We also care for inadvertent contamination of data sources, and ensure that all questions do not overlap with documents present in the training corpus.

Overall, our evaluation suite includes questions on: (1) design knowledge about general circuit design, including both analog and digital circuits; (2) EDA script generation in Python and TCL; (3) bug summarization and analysis; and (4) open-source HDL code generation (VerilogEval [20]). We compare the performance of ChipNeMo models against LLaMA chat

models, as illustrated in Fig. 2. Results showcase that the combination of DAPT, Domain Adaptive tokenizer, and model alignment contributes to up to 24.3% increase in accuracy on the chip design domain knowledge QA task and an average of 11.5% improvements across all benchmarks.

IV. CASE STUDY: RTL FORMAL VERIFICATION

In this section, we discuss the viability of LLMs adapted to the domain of VLSI chip design to be further applied to tasks in hardware formal verification (FV). As a preliminary case study, we first formulate a straightforward task of translating a natural language (NL) description of a formal property into concrete SystemVerilog Assertion syntax. Note that this task is conceptually analogous to those measuring capabilities in code generation for software [29] and HDL [20].

A. FV Benchmark Creation

While prior work have explored whether language models can perform FV tasks, there has been a lack of comprehensive, open-source benchmarks to serve as basis for quantitative analysis. Therefore, we first create an evaluation benchmark for the task of NL description to SVA code generation.

We collect formal verification testbenches that are: (1) verifying *canonical* unit-level modules in FV, namely FIFOs/Queues, Arbiters, Counters, FSMs, and simple memory blocks; and (2) are written and reviewed by experts in formal verification, to ensure that our examples are suitable as golden references. Our approach differs from prior work where the majority of their limited number of test cases are taken from larger sub-system level blocks [16], e.g. assertions for a Transaction Lookahead Buffer (TLB) in a CPU design, and thus the failures of language models in generating assertions are confounded with their inability to comprehend the complex DUT. Instead, we focus on simpler designs with a variety of well-established assertions—this enables our evaluation to carefully examine LLM capabilities in correlating high-level descriptions of formal properties to the given testbench context and produce correct SVA assertions.

Overall, our benchmark draws from 13 different testbenches and consist of a total of 81 human-crafted reference assertions. A summary is shown in Table II. From each reference, we hand-annotate natural language questions, based on which the language model is expected to infer the correct assertion implementation. We specify each test case at a high-level rather

Name	# Variations	# Assertions	Median LOC
IR1W FIFO	4	12	134
Multi-Port FIFO	1	6	251
Arbiter	4	46	70
FSM	2	5	168
Counter	1	5	152
RAM	1	7	99
Total	13	81	134

TABLE II: Summary of the NL to SVA benchmark suite. Each testbench is parameterized and the complexity of each property varies by the choice of parameters. For our evaluation, however, we measure LLM’s capability to generate SVA assertions that is independent of parameter selection.

than providing a description that is essentially an explicitly translation of the SVA syntax into English. An example is shown below:

Direct SVA-Translated NL Query:

Given the testbench, create an immediate SVA assertion that checks that `fifo_empty` and `rd_pop` are both never high in the same cycle.

High-Level NL Query (Ours):

Given the testbench, create an immediate SVA assertion that checks that the FIFO does not overflow, assuming there is no bypass.

Reference solution:

```
always @(posedge clk) begin
  asrt: assert ((fifo_empty && rd_pop) !== 1'b1);
end
```

Notice that compared to the first query that directly translates the reference SVA assertion into NL text, our query refers to the functionality of the property. This high-level query requires models to reason about the semantics of “overflow” in the context of a testbench verifying a FIFO unit.

B. Automated Evaluation Methodology

We again find the necessity for rapid and quantitative assessment of various LLMs. Here, we employ success rate in the form of Pass@ k as the performance metric, which is a widely-adopted measure of LLM code generation capabilities. In our case, we measure Syntax Pass@1 where we check if the LLM generated code snippet is syntactically correct. To administer this check, we append the reference testbench with LLM output code and use a SVA parser to check for syntax errors: as an open-source back-end, we employ SymbiYosys (sby) [30] that supports parsing immediate assertions. Accordingly we limit the test cases for this study to immediate assertions, where the expansion of our findings to concurrent assertions is left as future work. We also measure the BLEU score, a standard metric in evaluating similarity of long-form LLM outputs and reference solution, as a proxy for functional correctness. For our benchmark consisting of testbenches about relatively straightforward DUTs, the n-gram precision reflected in the BLEU scores gives high correlation with whether the model response is close to the correct solutions.

C. Experimental Results

We analyze the performance of ChipNeMo-70B against LLMs of similar model sizes on our FV evaluation benchmark. We consider two open-source models LLaMA-70b-chat and Mixtral-8x7B, a sparse Mixture-of-Expert (MoE) model. For full comparison, we also include two proprietary models, gpt-4 and gpt-3.5-turbo, with are much larger in size and are generally more capable than the models mentions above. We evaluate models in 1-shot and 3-shot settings, where in k -shot, the model is given an example of k pairs of NL query and reference solutions from a testbench different from that the model is being tested on. All model inference is performed with sampling temperature 0.

Model	Syntax Pass@1		BLEU	
	1-shot	3-shot	1-shot	3-shot
LLaMA2-70B-chat	0.765	0.741	0.390	0.385
Mixtral-8x7B	0.864	0.840	0.394	0.417
gpt-3.5-turbo	0.704	0.840	0.428	0.483
gpt-4	0.988	0.988	0.499	0.549
ChipNeMo-70B	0.851	0.889	0.417	0.424

TABLE III: LLM performance on the NL to SVA code generation task.

Results in Table III show that (1) gpt-4, the most capable LLM currently available, is capable of producing syntactically correct SVA assertions; and (2) ChipNeMo out-performs similarly sized open-source models and is competitive against gpt-3.5-turbo. The evaluation of BLEU scores indicate that, however, models need further improvement towards accurately translating NL descriptions into precise SVA implementations, and an important future research would be to develop adapted LLMs equipped with this capability.

V. CONCLUSION

In this work, we demonstrate the promises of domain-adapting LLMs, instead of using off-the-shelf models, towards problem solving and task automation in VLSI design. Further, from our case study on applying ChipNeMo to formal verification, we highlight that although general VLSI domain adaptation helps, fully grounding a model on the FV domain requires further, or possibly a new set of, pretraining and fine-tuning on FV-specific data. We anticipate that future work will train and develop better adapted language models for applications in FV, as well as investigate inference strategies towards improved LLM reasoning and understanding about formal properties of hardware designs.

VI. ACKNOWLEDGEMENTS

Authors deeply appreciate the support and technical guidance from Vigyan Singhal (NVIDIA).

REFERENCES

- [1] OpenAI, “Gpt-4 technical report,” 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:257532815>
- [2] H. Touvron and et al., “Llama 2: Open foundation and fine-tuned chat models,” 2023.
- [3] G. T. Google, “Gemini: A family of highly capable multimodal models,” *ArXiv*, vol. abs/2312.11805, 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:266361876>
- [4] A. Q. Jiang, A. Sablayrolles, A. Roux et al., “Mixtral of experts,” *ArXiv*, vol. abs/2401.04088, 2024. [Online]. Available: <https://api.semanticscholar.org/CorpusID:266844877>
- [5] J. Wei, X. Wang, D. Schuurmans et al., “Chain of thought prompting elicits reasoning in large language models,” *ArXiv*, vol. abs/2201.11903, 2022. [Online]. Available: <https://api.semanticscholar.org/CorpusID:246411621>
- [6] K. Singhal and et al., “Towards expert-level medical question answering with large language models,” *ArXiv*, vol. abs/2305.09617, 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:258715226>
- [7] K. M. Jablonka and et al., “14 examples of how llms can transform materials science and chemistry: a reflection on a large language model hackathon,” *Digital Discovery*, vol. 2, pp. 1233 – 1250, 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:259138569>
- [8] M. Liu, T.-D. Ene, R. Kirby et al., “Chipnemo: Domain-adapted llms for chip design,” *ArXiv*, vol. abs/2311.00176, 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:264833257>

- [9] J. Blocklove, S. Garg, R. Karri et al., “Chip-chat: Challenges and opportunities in conversational hardware design,” *2023 ACM/IEEE 5th Workshop on Machine Learning for CAD (MLCAD)*, pp. 1–6, 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:258833019>
- [10] S. Wu, O. Irsoy, S. Lu et al., “Bloomberggpt: A large language model for finance,” 2023.
- [11] A. Venigalla, J. Frankle, and M. Carbin, “BioMedLM: a domain-specific large language model for biomedical text,” 2022. [Online]. Available: <https://www.mosaicml.com/blog/introducing-pubmed-gpt>
- [12] S. Gururangan, A. Marasović, S. Swayamdipta et al., “Don’t stop pretraining: Adapt language models to domains and tasks,” 2020.
- [13] J. Wei, M. Bosma, V. Y. Zhao et al., “Finetuned language models are zero-shot learners,” 2022.
- [14] R. Kande, H. A. Pearce, B. Tan et al., “Llm-assisted generation of hardware assertions,” *ArXiv*, vol. abs/2306.14027, 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:259251452>
- [15] W. Fang, M. Li, M. Li et al., “Assertllm: Generating and evaluating hardware verification assertions from design specifications via multi-llms,” *ArXiv*, vol. abs/2402.00386, 2024. [Online]. Available: <https://api.semanticscholar.org/CorpusID:267364845>
- [16] M. Orenes-Vera, M. Martonosi, and D. Wentzlaff, “Using llms to facilitate formal verification of rtl,” *ArXiv*, vol. abs/2309.09437, 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:261884579>
- [17] H. A. Pearce, B. Tan, and R. Karri, “Dave: Deriving automatically verilog from english,” *2020 ACM/IEEE 2nd Workshop on Machine Learning for CAD (MLCAD)*, pp. 27–32, 2020. [Online]. Available: <https://api.semanticscholar.org/CorpusID:221445942>
- [18] E. Nijkamp, B. Pang, H. Hayashi et al., “Codegen: An open large language model for code with multi-turn program synthesis,” *ICLR*, 2023.
- [19] S. Thakur, B. Ahmad, H. A. Pearce et al., “Verigen: A large language model for verilog code generation,” *ArXiv*, vol. abs/2308.00708, 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:260379192>
- [20] M. Liu, N. Pinckney, B. Khailany et al., “VerilogEval: evaluating large language models for verilog code generation,” in *2023 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2023.
- [21] Z. He, H. Wu, X. Zhang et al., “Chateda: A large language model powered autonomous agent for eda,” *2023 ACM/IEEE 5th Workshop on Machine Learning for CAD (MLCAD)*, pp. 1–6, 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:261049069>
- [22] C. Hahn, “Towards improving verification productivity with circuit-aware translation of natural language to systemverilog assertions,” 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:259289057>
- [23] M. Hassan, S. Ahmadi-Pour, K. Qayyum et al., “Llm-guided formal verification coupled with mutation testing,” [Online]. Available: <https://api.semanticscholar.org/CorpusID:267393754>
- [24] S. J. Frederiksen, J. J. Aromando, and M. S. Hsiao, “Automated assertion generation from natural language specifications,” *2020 IEEE International Test Conference (ITC)*, pp. 1–5, 2020. [Online]. Available: <https://api.semanticscholar.org/CorpusID:231684619>
- [25] J. Zhao and I. G. Harris, “Automatic assertion generation from natural language specifications using subtree analysis,” *2019 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pp. 598–601, 2019. [Online]. Available: <https://api.semanticscholar.org/CorpusID:155108593>
- [26] M. Kang, A. Nova, E. Singh et al., “Lfps: Learned formal proof strengthening for efficient hardware verification,” *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, pp. 1–9, 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:265524521>
- [27] O. Kuchaiev, J. Li, H. Nguyen et al., “Nemo: a toolkit for building ai applications using neural modules,” 2019.
- [28] D. Hendrycks, C. Burns, S. Basart et al., “Measuring massive multitask language understanding,” 2021.
- [29] M. Chen, J. Tworek, H. Jun et al., “Evaluating large language models trained on code,” 2021.
- [30] YosysHQ, “Symbiosys (sby) – front-end for yosys-based formal verification flows,” <https://github.com/YosysHQ/sby>, 2024.